Informe técnico de compatibilidad y seguridad

Aplicación RoxCrew / MyShift - revisión para despliegue en Roxi/Roxer

Fecha del análisis: 27 de octubre de 2026
Alcance: código disponible en el repositorio local, configuración de ejecución, autenticación, autorización, persistencia, archivos, tiempo real y requisitos de la plataforma Roxer.

DICTAMEN

La aplicación puede ejecutarse como prototipo o staging, pero no está adaptada para un despliegue público con datos reales. Antes de producción deben corregirse los controles de autenticación, el secreto de IA, el almacenamiento de sesiones, la persistencia y la compatibilidad con el modelo serverless/WebSocket de Roxer.

Supuesto de plataforma

El término 'Roxi' se interpreta como Roxer, cuya documentación indica soporte para Node.js 20 y Express, despliegue dentro de Lambda, persistencia SQLite en /mnt/data y auto-sleep tras inactividad. Si se trata de otro servidor, debe sustituirse esta matriz por la documentación de ese proveedor.

Resumen ejecutivo

Área	Estado	Conclusión
Runtime Node/Express	OK	Compatible en principio con Node.js 20 y Express.
Persistencia	CRITICA	JSON en memoria y disco local; no es una base multi-instancia.
Uploads	ALTA	Archivos locales en public/uploads; requieren almacenamiento durable.
Socket.IO	ALTA	Debe confirmarse soporte de WebSocket en Lambda; la documentación consultada no lo garantiza.
Seguridad	CRITICA	Hay bypasses de autenticación, cuenta de prueba y secreto por defecto.
Build/runtime	MEDIA	El workspace no tiene node_modules y la ejecución usa tsx desde devDependencies.

1. Compatibilidad con Roxi/Roxer

La plataforma documenta soporte para aplicaciones Node.js/Express y despliegue serverless. La aplicación actual no es stateless: conserva estado en memoria, escribe JSON y archivos locales, y crea un servidor HTTP persistente para Socket.IO.

Componente	Situación actual	Compatibilidad	Adaptación requerida
Node/Express	server.ts usa Express y PORT.	**OK**	Fijar Node 20 y revisar start/build.
Frontend Vite	vite build genera dist.	**OK**	Confirmar SPA fallback y assets.
Base de datos	Map en memoria + data/database.json.	**CRITICA**	Migrar a SQLite en /mnt/data o PostgreSQL.
Uploads	public/uploads con multer diskStorage.	**ALTA**	Usar S3/R2/objeto durable o volumen confirmado.
Tiempo real	Socket.IO sobre http.createServer.	**ALTA**	Confirmar WebSocket; si no, usar polling/pub-sub o servidor persistente.
Secretos	JWT_SECRET y CLAUDE_API_KEY por environment.	**ALTA**	Configurar variables en Roxer; eliminar defaults.
Cold start	Estado inicial y DB se cargan al arrancar.	**MEDIA**	Inicialización idempotente y sin seed de cuentas reales.

Referencia de plataforma: documentación de Roxer, secciones de runtime, Lambda, SQLite y planes de hosting.

2. Hallazgos de seguridad y correcciones

ID	Severidad	Hallazgo y evidencia	Corrección obligatoria
SEC-01	**CRITICA**	Bypass en **server/middleware.ts:37-51**: headers x-user-email/x-user-id y usuario admin por defecto autenticaban sin JWT.	Eliminar ambos fallbacks. requireAuth debe aceptar solo cookie HttpOnly o Bearer validado.
SEC-02	**CRITICA**	Cuenta de prueba y contraseñas conocidas en authRoutes.ts:200-212 y seed de db.ts.	Eliminar seed de producción, forzar cambio de credenciales y rotar todo secreto.
SEC-03	**CRITICA**	aiRoutes.ts:16 usa roxcrew_claude_secret_2026 como fallback; el endpoint no requiere sesión.	Sin default; exigir API key desde secret manager, rate limit y autorización del board.
SEC-04	**ALTA**	El JWT se devuelve en JSON y se guarda en localStorage/sessionStorage en src/lib/api.ts:14-21.	No devolver token al frontend; usar cookie HttpOnly.
SEC-05	**ALTA**	Cookie SameSite=None en auth.ts:58-65, sin protección CSRF visible.	Usar SameSite=Lax/Strict si es same-origin; añadir CSRF/origin checks si se requiere cross-site.
SEC-06	**ALTA**	CORS Socket.IO origin:true en socket.ts:33-38.	Lista allowlist de dominios HTTPS autorizados; rechazar cualquier otro origen.
SEC-07	**ALTA**	Invitaciones en invitationRoutes.ts:173-222 validan rol global, pero no siempre tenant/board propietario.	Centralizar autorización de tenant y board antes de leer/revocar tokens.
SEC-08	**MEDIA**	JSON body de hasta 50 MB y uploads basados en MIME del cliente.	Limitar por endpoint, validar esquema, inspeccionar magic bytes y rechazar contenido no permitido.

3. Persistencia, datos y operación

El servidor carga todo en Maps y serializa a un único JSON. En un entorno serverless puede haber varias instancias y cada una puede tener una copia distinta del estado.

- Migrar usuarios, boards, listas, tarjetas, miembros, invitaciones y actividad a SQLite usando /mnt/data solo si Roxer confirma ese volumen; preferiblemente PostgreSQL administrado para crecimiento.

- Eliminar data/database.json del repositorio y crear un proceso de migración con backup cifrado.

- Reemplazar saveToDisk por transacciones y constraints: email único, relaciones, expiración de invitaciones y control de concurrencia.

- Mover avatares, fotografías y wallpapers a almacenamiento de objetos con URLs privadas o URLs firmadas y expiración.

- No servir uploads directamente desde el filesystem de la aplicación. Mantener autorización por board al emitir la descarga.

- Añadir backups automáticos, retención, restauración probada y logs de auditoría para permisos y membresías.

- Definir health check, métricas de errores, latencia, cold starts y almacenamiento sin revelar datos.

4. Tiempo real y arquitectura

Socket.IO funciona bien en un proceso Node persistente, pero la documentación revisada de Roxer describe una función Lambda aislada por aplicación. No debe asumirse que una conexión WebSocket y activeBoardUsers sobrevivan o se compartan entre invocaciones.

- Confirmar con Roxer soporte oficial para WebSocket/Socket.IO, timeout, upgrade y sticky sessions.

- Si no hay soporte, cambiar presencia y eventos a polling, SSE soportado o un proveedor pub/sub externo.

- No mantener presencia en memoria local; usar Redis, DynamoDB o base persistente si la función escala.

- Probar dos instancias simultáneas: un usuario cambia una tarjeta y otro recibe el evento.

- Controlar reconexión, duplicados, orden de eventos e idempotencia.

5. Plan detallado de corrección

Fase	Trabajo	Resultado verificable
P0 Seguridad	Eliminar bypasses, cuenta de prueba, defaults, token en storage y corregir cookie/CSRF.	Sin JWT, headers falsos y origen externo responden 401/403.
P0 Datos	Migrar JSON y uploads a almacenamiento durable; separar seed demo de producción.	Reinicio y redeploy no pierden datos; dos instancias ven la misma información.
P0 Runtime	Fijar Node 20, compilar backend TypeScript a JavaScript y usar node como start.	npm ci, npm run build y npm start funcionan en una imagen limpia.
P1 API	Validación por esquema, límites por endpoint, rate limits, errores uniformes y CORS allowlist.	Suite API pasa casos válidos, inválidos, abuso y permisos por tenant.
P1 Archivos	Validar contenido real, nombres seguros, tamaño, antivirus opcional, URL firmada y limpieza.	No se acepta HTML/SVG ejecutable ni payload disfrazado de imagen.
P1 Tiempo real	Confirmar WebSocket o cambiar arquitectura; probar reconexión y multi-instancia.	Eventos consistentes después de cold start.
P2 Operación	CI/CD, SAST, dependency audit, backups, alertas, rollback y dominio HTTPS.	Release reproducible, auditable y con rollback probado.

6. Checklist previo a producción

Control	Criterio de aprobación	Estado actual
Secretos	Todos en variables de Roxer; ningún valor por defecto ni en Git.	**PENDIENTE**
Autenticación	No existen bypasses; cookies seguras; sesiones revocables.	**PENDIENTE**
Autorización	Tests de tenant, board, rol y acceso a archivos.	**PENDIENTE**
Persistencia	DB y archivos sobreviven reinicio, redeploy y concurrencia.	**PENDIENTE**
Build	Build limpio y arranque con Node 20, sin herramientas de desarrollo en runtime.	**PENDIENTE**
WebSocket	Soporte confirmado o alternativa implementada.	**PENDIENTE**
Pruebas	Lint, unit, integración, seguridad y smoke test post-deploy.	**PENDIENTE**
Backups	Backup cifrado y restauración validada.	**PENDIENTE**

7. Configuración recomendada de despliegue

La configuración exacta depende de la cuenta Roxer, pero el mínimo esperado es:

Variables de entorno requeridas

```
NODE_ENV=production
JWT_SECRET=<secreto aleatorio de alta entropía>
CLAUDE_API_KEY=<solo si el endpoint se mantiene>
DATABASE_URL=<si se usa PostgreSQL>
STORAGE_BUCKET=<bucket privado>
APP_ORIGIN=https://<dominio oficial>
```

Reglas de publicación

- No desplegar data/database.json ni credenciales demo.

- No ejecutar npm start con tsx desde devDependencies; compilar y arrancar con node.

- Usar HTTPS y dominio propio solo después de validar cookies, CORS y WebSocket.

- Usar un entorno staging separado del entorno de producción.

- Realizar smoke tests para login, board, invitación, upload, permisos y reconexión.

- Revisar logs de Roxer sin registrar passwords, JWT, cookies, API keys ni tokens de invitación.

8. Resultado de las verificaciones locales

La revisión estática encontró los hallazgos descritos. Las verificaciones de build no pudieron completarse porque node_modules no está instalado: npm run lint y npm run build devolvieron tsc: command not found y vite: command not found. npm audit tampoco pudo ejecutarse porque no existe package-lock.json; el proyecto contiene bun.lock.

Conclusión final

No recomiendo publicar esta versión con usuarios reales. Recomiendo un ciclo de hardening P0, migración de datos, validación de WebSocket en Roxer y staging aislado. Una vez completado el checklist, la aplicación puede ser candidata a producción en Roxer si la plataforma confirma persistencia y tiempo real adecuados.

Fuentes consultadas

- Documentación de Roxer: https://roxer.com/docs/ (runtime Node.js 20, Express, Lambda, SQLite en /mnt/data, planes, variables de entorno y dominios).
- Código local analizado: server.ts, server/auth.ts, server/middleware.ts, server/routes/authRoutes.ts, server/routes/aiRoutes.ts, server/routes/invitationRoutes.ts, server/socket.ts, server/db.ts y src/lib/api.ts.